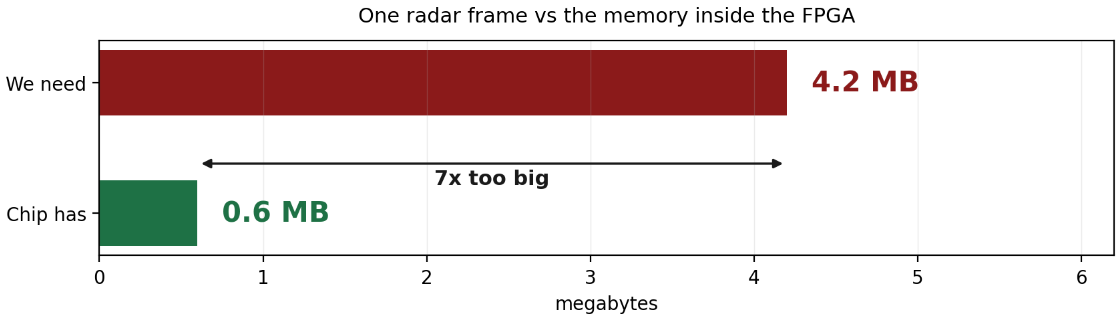
The Memory Problem

Why our radar data does not fit on the chip,
and why changing boards does not help

The problem in one line

Our radar data is 4.2 MB. The memory inside the FPGA chip is 0.6 MB. It does not fit, and it is not close — we are seven times over.



What the 4.2 MB actually is

It is **one radar frame** — everything the radar collects in 10.24 milliseconds, which all has to be held in memory at the same time.

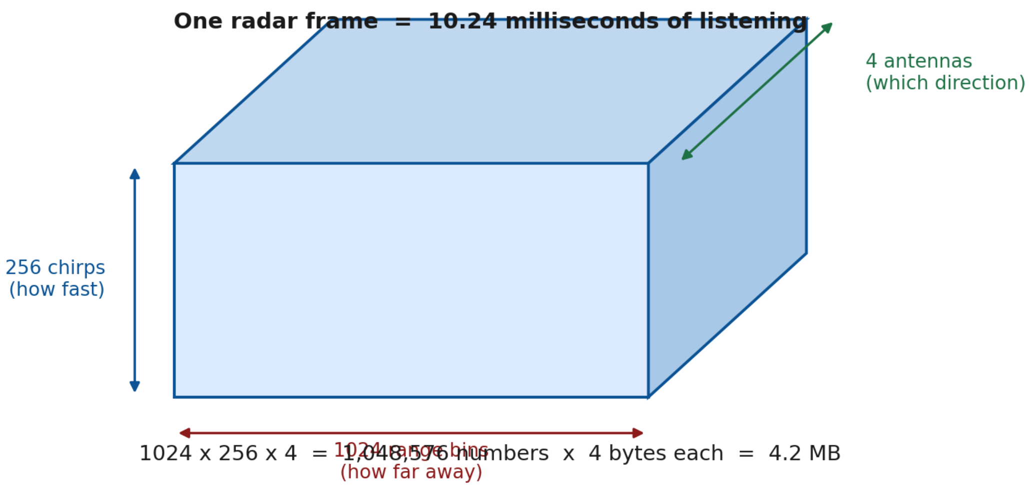


Figure 1 — One frame of radar data.

Where the number comes from

In one frame the radar sends 256 chirps, one after another. During each chirp it takes 1024 samples. All four antennas listen at the same time.

```
1024 samples   x  256 chirps   =    262,144
  262,144      x  4 antennas   =  1,048,576 numbers

each number = 16-bit real + 16-bit imaginary = 4 bytes

1,048,576  x  4 bytes          =  4,194,304 bytes  =  4.2 MB
```

Each number is a single measurement: *at this distance, during this chirp, on this antenna — what came back.* It has two parts because we need both the strength of the signal and its direction.

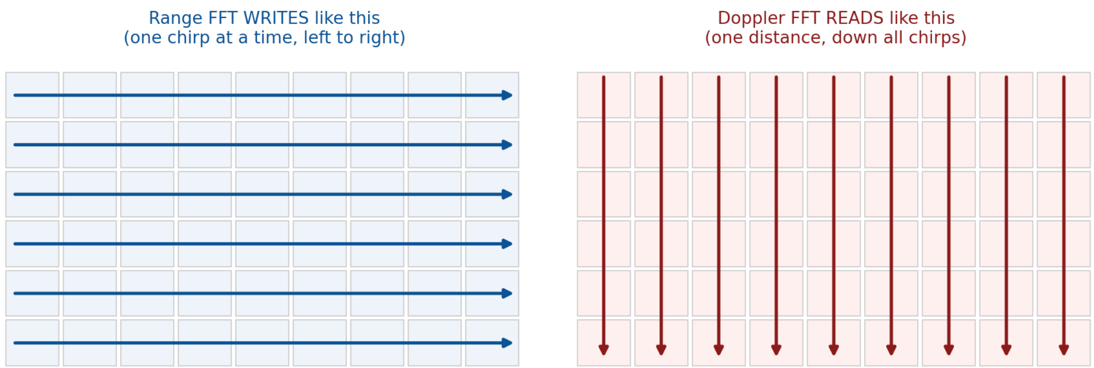
And the chip:

```
140 block RAM tiles   x  36 Kbit  =  4.9 Mbit  =  0.6 MB
```

That memory is built into the silicon. It cannot be increased.

Why we cannot just keep a small piece

This is the part that surprises people. The problem is not the amount of data by itself — it is that **the order we produce it in does not match the order we need it in.**



The orders do not match — so everything must be stored before the second FFT can start

Figure 2 — The range FFT fills the table row by row. The Doppler FFT needs to read it column by column.

The range FFT works **one chirp at a time**. It finishes chirp 1 completely, then chirp 2, and so on. But the Doppler FFT needs **one distance across all 256 chirps** — a whole column.

So the Doppler FFT cannot start until every single chirp has been through the range FFT. All 1,048,576 numbers have to sit somewhere in the meantime. **That waiting area is the corner turn, and it cannot be avoided.**

Why moving to the PYNQ-Z2 does not help

We looked at the PYNQ-Z2 as an alternative. It does not solve this problem, for a simple reason:

Zybo Z7-20	ZYNQ **XC7Z020**-1CLG400C
PYNQ-Z2	ZYNQ **XC7Z020**-1CLG400C

They use exactly the same FPGA chip.

The 0.6 MB of fast memory sits *inside* the FPGA, not on the circuit board. Same chip means the same memory, down to the bit. No board can change it.

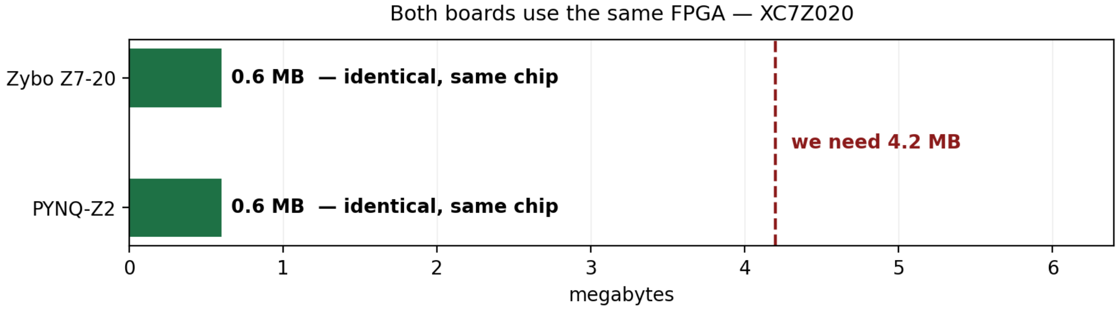


Figure 3 — Same chip, same limit, on both boards.

What is different between the boards

The boards do differ — but only in the **separate DDR memory chip** sitting next to the FPGA:

	Zybo Z7-20	PYNQ-Z2
Memory inside the FPGA	0.6 MB	0.6 MB (same)
DDR memory on the board	1 GB	512 MB
DDR connection width	32-bit	16-bit
How fast we can move data	~4.3 GB/s	~2.1 GB/s

So the PYNQ-Z2 is not an upgrade for us. It has the same on-chip limit, and its connection to the big memory is only **half as wide** — which matters, because once the data goes to DDR we have to move it back and forth constantly.

Where this leaves us

Putting it together:

- The data must be stored somewhere between the range FFT and the Doppler FFT. This is forced by the order the data arrives in, not by a design choice.
- It is 4.2 MB. The FPGA holds 0.6 MB. It will not fit on the chip.
- Both boards we have use the same FPGA, so neither board solves it.
- Therefore the data has to go out to the DDR memory on the board and come back.

DDR has plenty of room — 4.2 MB into 512 MB or 1 GB is nothing. **The new question is speed:** can we move the data in and out fast enough to keep up with the radar? That is what we need to work out next.

The one thing to remember

The FFT engine itself is small — about a tenth of the chip. The memory needed between the stages is thirty times bigger than the engine, and does not fit on the chip at all.

In this project the memory is the hard problem. The maths is the easy part.

Solutions to be discussed separately.